With streaming implemented, we can now explore some other important features of AI web apps, such as supporting more than one AI provider and being able to swap them on the fly.

3.3 Working with multiple AI providers

So far, we've been working with a single AI model for the AI web application example. However, since the AI landscape is evolving and we have access to multiple AI models, it's useful to be able to adopt and swap providers based on our specific needs. For example, some AI models excel at certain tasks like code generation, but others are good at generating poems. Wouldn't it be great if we could create tools that can employ all models depending on the user's needs?

As you may have guessed, to achieve that, we need to create abstractions, since integrating and managing multiple AI providers and models without effort poses a challenge. We don't want to hardcode every provider API out there, since each provider offers unique capabilities, strengths, and APIs, and that would make the application less flexible and harder to maintain.

In this section, we'll explore how the Vercel AI SDK simplifies working with multiple AI providers and models, enabling developers to focus on building powerful AI applications.

3.3.1 Handling different AI providers and models

Given the explosion of LLMs and AI technologies, every company invested in this space wants to offer its own solution. For example, AI providers, such as OpenAI, Anthropic, Google, and others, have a wide range of language models with varying capabilities and APIs. Each one offers a client or an API that interacts with its models, but there is no requirement to have a common API between them. This creates friction when it comes to integrating multiple clients' various providers into a single application.

For example, consider a scenario where you want to generate text using both OpenAI's GPT-4 and Cohere models. Without a standardized approach, you would need to handle each provider's specific API requirements separately.

Listing 3.9 Calling both OpenAI and Cohere clients

```
import OpenAI from 'openai';
import 'dotenv/config';
const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});
const openaiResponse = await openai.chat.completions.create({.
  model: 'gpt-3.5-turbo',
  messages: [{ role: 'user', content: 'What is 7+7 and why?' }],
  max_tokens: 150,
});

import { CohereClient } from "cohere-ai";
const cohere = new CohereClient({.
    token: process.env.COHERE_API_KEY,
```

Imports, configures, and uses the OpenAI chat completion functionality

Creates an instance of the Cohere client that interfaces with Cohere AI

max_tokens is a mandatory parameter for the Open AI chat completions API.